

Tutor Mx System

Gitea Sprint Handbook and Review Guide

Beginner-friendly Sprint 1-4 issue, testing, handover and deployment guide
for six Members

CURRENT STACK

React + Vite frontend | Auth0 | handwritten Node.js + Express API | Prisma | Neon PostgreSQL | Gitea main | GitHub
deployment mirror | Codecov | Cloudflare Pages | Render API

COMS3011A Project 1 | 2026

Fully updated 11 September 2026

AI Declaration: This edition was updated with ChatGPT [GPT-5.6 Sol] from the supplied handbook, the current integrated Tutor Mx source and the team's recorded workflow. The team remains responsible for checking requirements, commands and evidence before submission.

UPDATED THROUGH 16 SEPTEMBER 2026

Sprint 1-2 history retained; post-Sprint 2 stabilisation and Sprint 3-4 roadmap revised.

What changed in this edition

- Gitea main is the official shared code. The old handbook instructions that told Members to target a develop branch are removed.
- All six Sprint 1 Member contributions are treated as the current integrated baseline. Sprint 2-4 work builds on that baseline instead of recreating it.
- New users choose Student, Tutor or Organiser before Auth0 sign-up. Express performs server-side onboarding and creates/links the matching Neon Profile automatically.
- Existing users sign in once and are routed directly to their saved role workspace. Normal sign-in does not ask them to choose a role again.
- Organiser Tutor Management now lists self-registered Tutor accounts. Organisers search by name/email and Edit the database-backed profile; they never type an Auth0 subject/User ID.
- The exact current repository test gate (npm run check), coverage commands and local browser verification steps are documented on pages 7-9.
- The lecturer-runner rule is updated: two shared Gitea Action runners are now available to all groups; queued jobs are normal, local tests remain required, and completed CI evidence must be kept.
- The deployment path is clarified: approved Gitea main -> GitHub deployment mirror -> fresh Vite build -> manual Cloudflare Production upload for the frontend.

CURRENT SPRINT 1 BASELINE

Members 1-6 are integrated in the current main-ready source. The Organiser/Tutor features use real API routes where implemented; Student overflow/volunteer remains the deliberate Sprint 1 mock boundary until Sprint 2 connects the real Student API. The Sprint 1 Save allocation control is still a planned placeholder until the allocation-write endpoint is completed.

1 SEPTEMBER RUNNER UPDATE

Shared runners: sdp-runner-1 and sdp-runner-2. They are shared across groups, so jobs may queue and then auto-process. Each runner is limited to 4 GB memory and currently a 3-hour timeout. A private group runner is optional, not required.

How the team works now

Use this method in Sprint 1, 2, 3 and 4

NO DEVELOP BRANCH

Gitea main is the official shared branch. Members do normal feature work on their own branch. The Integration Lead reviews/integrates approved work into main.

Step	What to do
1. Refresh	git switch main -> git pull origin main
2. Create/use your branch	git switch -c SX/mY/short-name (or switch to your existing issue branch)
3. Work locally	Make only the changes for your issue. Keep secrets out of files/screenshots.
4. Test	Follow page 7. Then run the browser/local verification on page 8.
5. Commit your branch	Commit focused work on your branch. Do not commit directly to main.
6. Push + hand over	Push the branch to Gitea and tell the group: issue ID, branch, tests run, known limits, AI attribution.
7. Review + integrate	Reviewer checks the issue. Integration Lead integrates approved work into main and reruns the affected regression/full gate.
8. Refresh again	After main changes, everyone pulls the latest main before starting/resuming later work.

INTEGRATION LEAD EXCEPTION

When the Integration Lead performs integration/repository-maintenance work themselves, reviewed/tested changes may be committed through their integration workflow to main. This does not turn main into a normal Member working branch.

NEW SHARED-RUNNER STEP AFTER PUSH

After a Member pushes the branch, confirm the Gitea Actions workflow starts. If it is queued, wait for a shared runner to become free. Keep the completed run link/status with the handover evidence. The reviewer and Integration Lead check the CI result as part of review; local page 7-8 testing is still required.

Architecture and automatic role onboarding

Layer	Beginner meaning
1. Browser	Loads Tutor Mx. New user chooses Student / Tutor / Organiser.
2. Cloudflare Pages	Serves the React + Vite frontend. Only public VITE_* configuration belongs here.
3. Auth0	Universal Login handles Google or email/password identity and email verification.
4. React onboarding	First-time account calls POST /api/auth/onboarding with the selected role + bearer token.
5. Express on Render	Verifies Auth0 identity, applies role policy and uses server-only Auth0 Management credentials when needed.
6. Prisma + Neon	Creates/reads the matching Profile and application data. Browser code never connects directly to Neon.
7. Later sign-ins	React calls /api/me and opens the saved role workspace automatically.
ONE ROLE PER TUTOR MX ACCOUNT The selected role is treated as one-time. Use separate test accounts/emails for Student, Tutor and Organiser. Do not use normal onboarding to switch an established account between roles.	

SECRETS RULE

DATABASE_URL and Auth0 Management API client secrets stay on the backend host. They must never be placed in VITE_* variables, screenshots, issues, chat, or committed files.

Very beginner guide: create an account or sign in

Action	Expected path
Create Student	Choose Student -> Create Student account -> Auth0 -> server onboarding -> /student
Create Tutor	Choose Tutor -> Create Tutor account -> Auth0 -> server onboarding -> /tutor
Create Organiser	Choose Organiser -> Create Organiser account -> Auth0 -> server onboarding -> /organiser
Existing account	Click Sign in -> Auth0 -> /api/me -> correct saved workspace. No second role choice.
Switch test role	Sign out first, then use the separate account created for the other role.
EMAIL VERIFICATION If Auth0 requires verification for an email/password account, verify the email and return to Tutor Mx. An unverified account must not be silently assigned a role.	

NORMAL SETUP IS AUTOMATIC

Do not manually create Neon Profile rows or manually assign individual Tutor Mx roles in Auth0 for ordinary new-user sign-up. Manual admin intervention is troubleshooting, not the normal Member workflow.

Organiser Tutor Management - current professional method

WHAT THE ORGANISER SEES

Search for a tutor by name or email. Select Edit to manage the tutor's details and weekly-hour limit.

- Tutor accounts appear after the Tutor signs up through the normal self-service Tutor flow.
- The Organiser does not create the Tutor identity and does not copy an Auth0 User ID/subject.
- The UI uses the tutor's internal database record automatically when the Organiser edits details.
- Marks and busy times are managed from the selected Tutor record.
- Technical explanations about Auth0/Neon are documentation concerns; they should not clutter the normal organiser interface.

SPRINT PLANNING CONSEQUENCE

Sprint 2 must not rebuild a manual tutor-create/Auth0-subject form. Sprint 2 hardens the registered-tutor management flow and completes allocation writes/approvals.

How to run the tests in VS Code / PowerShell

Every Member uses this page before handing work over

FAST RULE

From the repository root, npm run check is the current full local release-style gate. It runs Prisma generation, lint, backend type-checking, backend + frontend tests, production builds and the repository security check.

```
npm ci
npm ci --prefix frontend
npm run generate
npm run check
npm run coverage
```

Command	What it proves
npm run check	Full gate: generate -> lint -> typecheck -> tests -> build -> security check.
npm test	Backend + frontend automated tests.
npm run test:backend	Backend Vitest suite only.
npm run test:frontend	Frontend Vitest suite only.
npm run coverage	Backend + frontend coverage reports.
npm run lint	Backend + frontend lint.
npm run typecheck	Backend TypeScript check.
npm run build	Production backend + frontend builds.
npm run security:check	Tracked-file secret / repository safety check.

WHAT COUNTS AS PASSING

The command must exit successfully. Warnings are not the same as failed tests, but new warnings caused by your change should still be investigated and mentioned in the handover. Never hide a failed command.

AFTER THE LOCAL GATE PASSES

Commit and push the Member branch to Gitea, then confirm its Actions workflow runs on the shared lecturer runners. A queued job is not a failure; wait for it to auto-process. Save the run link/logs and Codecov result with the handover evidence.

How to check your work locally before the Integration Lead touches main

PURPOSE

This is the beginner smoke test Members run on their own branch. It answers: does my change actually work in the app before I tell the Integration Lead it is ready?

```
git switch main
git pull origin main
git switch <your-issue-branch>

# first clean setup / when dependencies changed
npm ci
npm ci --prefix frontend
npm run generate

# start frontend + backend together
npm run dev
```

- [] Open <http://localhost:5173>. The API normally runs at <http://localhost:3000>.
- [] Use the role/account relevant to your issue. For role testing, use separate Student, Tutor and Organiser test accounts.
- [] Repeat the acceptance path yourself: success case, invalid/failure case, and one relevant empty/permission case.
- [] Refresh the page and retry the action. Check that loading, error and success states remain usable.
- [] For protected flows, confirm another role cannot perform the action just by typing a URL or changing a request.
- [] Stop the dev servers with Ctrl+C. Then run `npm run check` and `npm run coverage`.
- [] Run `git status --short` and confirm only the files you intended to change are listed.

HANDOVER EVIDENCE

Send the issue ID + branch name + local commands/results + screenshot/API example where useful + known limitation. After pushing, also send the Gitea Actions run link/status and Codecov result once the shared runner finishes. Do not send secrets or production personal data.

AFTER PUSHING TO GITEA

Queued = normal on the shared runners; wait. Failed = investigate and rerun. Genuine runner/infrastructure outage = record the blocker plus local evidence and retry/contact staff. Never report CI as green unless the workflow actually completed successfully.

Lecturer update: shared Gitea Actions runners and CI evidence

WHAT CHANGED ON 1 SEPTEMBER 2026

The lecturer has set up two tested shared runners, sdp-runner-1 and sdp-runner-2, for all groups. Because they are shared, a job may sit in the queue and then auto-process when a runner becomes free. Local testing remains required.

WHAT WE MUST DO

- Before handover, run the page 7 automated checks and page 8 local acceptance/smoke checks.
- Push the Member branch to Gitea and let the Actions workflow run on the shared runners.
- If the job is queued, wait; queueing is expected when another group is using the runners.
- When the job completes, keep the run link/logs and Codecov result as CI evidence.
- If the workflow fails, investigate the code/configuration and rerun it. If the runner infrastructure genuinely remains unavailable, attach local evidence, record the blocker honestly and contact staff for persistent/complex runner issues.
- Do not weaken/remove tests and do not claim CI or Codecov is green when the workflow did not complete.
- A private runner is optional. If the group registers one, use a descriptive name such as <group-name>-runner-`<id>`; the lecturer may remove private runners for admin/security reasons.

RUNNER LIMITS AND QUEUE RULE

Shared runners are limited to 4 GB memory and currently time out after 3 hours; the lecturer may reduce the timeout to 30 minutes later. Keep workflows reasonably simple. Queued is not failed - wait for auto-processing.

Member handover -> Integration Lead -> Gitea main -> GitHub mirror

```
git status
git add <only-your-files>
git diff --cached --check
git commit -m "<clear issue-focused message>"
git push -u origin <your-issue-branch>
```

- Member: do not merge your own normal feature branch into main. Run local checks first, commit, and push the issue branch.
- After push: confirm Gitea Actions starts. If queued, wait for a shared runner; once complete, attach the run link/status and Codecov result to the handover.
- Reviewer: reproduce the issue acceptance checks, inspect changed files, and check both local evidence and the completed CI result.
- Integration Lead: pull latest main, integrate approved work, resolve conflicts with the author, rerun npm run check (plus coverage/database checks when relevant), then push Gitea main and verify the main-branch Actions result if the workflow triggers there.
- Deployment mirror: only after Gitea main is approved, sync that source to the GitHub deployment repository. GitHub is not the team source of truth.

BEFORE EVERY LATER TASK

After an integration changes main, pull main again before continuing Sprint 2/3/4 work. This prevents members from building on an old snapshot.

Manual Cloudflare Production deployment after approved integration

FRONTEND RELEASE PATH

Gitea main -> GitHub deployment mirror -> fresh local Vite build -> Cloudflare Pages Production upload.

```
git switch main
git pull origin main
cd frontend
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue
npm run build
```

- [] Open Cloudflare Dashboard -> Workers & Pages -> tutor-mx -> Create deployment.
- [] Choose Production.
- [] Upload the newly generated frontend/dist folder.
- [] Confirm all files upload successfully, then select Save and deploy.
- [] Open tutor-mx.pages.dev and hard refresh (Ctrl + Shift + R).
- [] Run the affected role smoke test on the live site.
- [] Backend/API changes must also be deployed to the hosted Express service using the team's approved deployment mirror/host workflow.

VITE RULE

VITE_* values are embedded at build time. If production frontend environment values change, rebuild dist before uploading it.

AI policy and Global Definition of Done

AI policy

- [] Remove secrets, tokens and personal data before using an AI tool.
- [] Read, understand and test AI-generated code before committing it.
- [] If a commit contains AI-generated code, use the course-required Assisted-by: <tool>[<model>] attribution with the actual tool/model.
- [] Keep README/document AI declarations current. Never invent test evidence, stakeholder evidence or tool/model names.

Global Definition of Done - applies to every issue in Sprint 1-4

- [] The issue-specific acceptance checks pass.
- [] Page 8 local browser/behavior check is completed for the affected flow.
- [] Page 7 automated test gate is run; relevant coverage is produced.
- [] Follow page 9: wait for queued shared-runner CI, keep completed run evidence, and document genuine infrastructure blockers honestly.
- [] Only intended files are committed on the Member branch; no secrets/personal data are added.
- [] The branch is pushed to Gitea and the assigned reviewer/Integration Lead is told it is ready.
- [] The Member does not merge normal feature work to main. Integration Lead integrates approved work and reruns regression.
- [] AI attribution/declarations are correct.

Sprint 1: plan, order and handovers

3-25 August 2026 | Integrated foundation, public demo and one secure vertical path

SPRINT 1 IS THE BASELINE

Current main already contains the integrated Member 1-6 Sprint 1 foundation. Use these issue cards for traceability, regression and follow-up fixes; do not rebuild working foundations.

- Keep the automatic Auth0 -> Express onboarding -> Neon Profile method as the normal account path.
- Keep Organiser Tutor Management registered-tutor based; no Auth0 subject field.
- Keep Tutor routes own-identity based; no browser tutor-ID ownership.
- Keep Student overflow/volunteer on the Sprint 1 mock boundary until Sprint 2 real endpoints are integrated.
- Keep Save allocation as the Sprint 1 placeholder until Sprint 2 allocation-write work is ready.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

EVERY MEMBER

Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.

Sprint 1 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M1-1 | Self-service Auth0 sign-up, role onboarding and direct routing

Branch: s1/m1/auth-entry | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Goal: As a new or returning user, I want Tutor Mx to link my authenticated identity to exactly one role and open the correct workspace without manual setup.

Acceptance checks

- A new user chooses Student, Tutor or Organiser before Auth0 sign-up; the server creates the matching Tutor Mx role/Profile automatically.
- An existing linked user signs in once and is routed directly to /student, /tutor or /organiser without choosing a role again.
- An unverified email or onboarding conflict is handled safely; passwords and Management API credentials never enter Tutor Mx browser code.

Member tasks

- [] Maintain the role-choice entry screen and Auth0 Universal Login flow.
- [] Use POST /api/auth/onboarding for first-time linking and /api/me for later sign-ins.
- [] Keep sign-out available from all three workspaces and cover role-routing failures with tests.

Handoff: Give Member 5 the bearer-token shape and give Members 2-4 the signed-in user/role interface.

S1-M1-2 | Account settings, password reset, deletion and route protection

Branch: s1/m1/auth-lifecycle | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1 and role names Organiser/Tutor/Student

Goal: As a Tutor Mx user, I want safe account recovery and protected role navigation so that I can manage access without exposing private data.

Acceptance checks

- Password-reset requests use a safe non-enumerating response and Auth0 owns the actual password workflow.
- Account deletion requires confirmation and the privileged Auth0/database deletion runs only on the backend.
- Wrong-role or unlinked access is blocked even if a user manually types a protected URL.

Member tasks

- [] Maintain Account Settings reset/deletion actions and confirmation states.
- [] Keep route guards aligned with backend role/ownership checks.
- [] Test cancel/confirm, wrong-role, missing-profile and signed-out paths.

Handoff: Give Member 5 the role matrix and give the Project Coordinator screenshots/test evidence for the security guide.

S1-M1-3 | Authentication contract and permission tests

Branch: s1/m1/auth-contract-tests | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1, S1-M1-2 and S1-M5-2

Goal: As the team, we want one tested bearer-token and role contract so that every feature uses the same secure identity rules.

Acceptance checks

- Protected routes return 401 for missing/invalid tokens and 403 for valid users with the wrong role.
- A permitted token reaches the handler only when its Tutor Mx role matches the linked Neon Profile.
- Fresh first-login onboarding is handled safely even when the first token predates the newly assigned role.

Member tasks

- [] Maintain the shared bearer-token request helper.
- [] Keep backend Auth0 issuer/audience/JWKS/expiry checks covered.
- [] Add contract tests whenever a new protected route is introduced.

Handoff: Member 5 consumes the token helper/fixtures; Members 2-4 use the request wrapper rather than copying auth code.

Sprint 1 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M2-1 | Organiser workspace and allocation-board foundation

Branch: s1/m2/organiser-board-shell | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Shared navigation from S1-M4-2; mock candidate contract from

Goal: As an organiser, I want one clear workspace for courses, tutors and allocation checks so that management work is easy to follow.

Acceptance checks

- The Organiser workspace loads only for the ORGANISER role and keeps loading/empty/error states usable.
- Candidate mark, clash and hour results are visible together before selection.
- The workspace remains keyboard reachable and usable on common phone widths.

Member tasks

- [] Maintain the Organiser navigation and allocation-board shell.
- [] Keep candidate data and rule-result components aligned with the API contract.
- [] Preserve clear loading, empty, error and retry states.

Handoff: Give Member 5 a list of required fields and give Member 4 any reusable table/card needs.

S1-M2-2 | Course management and registered-tutor management UI

Branch: s1/m2/course-tutor-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and S1-M5-2 contract

Goal: As an organiser, I want to manage courses and already-registered tutors using normal account information instead of technical Auth0 identifiers.

Acceptance checks

- Signed-up Tutor accounts appear from the database automatically and can be searched by name or email.
- The Organiser never has to enter, copy or see an Auth0 subject/User ID in the normal Tutor Management UI.
- Edit uses the internal tutor database record to maintain details, weekly-hour limit, marks and busy times.

Member tasks

- [] Maintain course add/edit/remove UI with validation and confirmations.
- [] Maintain tutor search/list/edit UI for self-onboarded Tutor profiles.
- [] Keep the interface professional: no implementation-level Auth0/Neon instructions in normal organiser screens.

Handoff: Member 6 uses the final field list in migrations; Member 5 matches the request/response shapes.

S1-M2-3 | Immediate allocation-rule messages and organiser UI tests

Branch: s1/m2/allocation-rule-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and rule outputs from S1-M5-3

Goal: As an organiser, I want mark, timetable and remaining-hour checks explained before saving so that invalid choices are obvious.

Acceptance checks

- Mark threshold, clash and remaining-hours checks each show written Pass/Fail or controlled unknown wording.
- Save remains unavailable until the selected tutor passes the required checks.
- Colour is never the only way a rule result is communicated.

Member tasks

- [] Maintain the reusable rule-result component.
- [] Test pass/fail/unknown and boundary values.
- [] Keep candidate refresh and selected-tutor state stable after errors.

Handoff: Member 5 supplies stable reason codes; Member 2 maps those codes to plain language without duplicating business logic.

Sprint 1 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M3-1 | Tutor dashboard using the signed-in tutor identity

Branch: s1/m3/tutor-dashboard | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2 and shared states from S1-M4-2

Goal: As a tutor, I want my own assignments and weekly-hour summary without entering a tutor ID so that another tutor cannot be selected by changing a URL.

Acceptance checks

- GET /api/tutor/dashboard derives the tutor from the verified token subject and returns only that tutor's data.
- A newly onboarded Tutor with no hour limit or allocations gets a usable no-work state rather than an error.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Maintain the protected tutor dashboard route and cards/table.
- [] Display unset weekly limits safely and never request another tutor by browser-supplied ID.
- [] Test own-data, no-work and failed-load cases.

Handoff: Give Member 5 the dashboard fields and Member 6 the query/seed-data needs.

S1-M3-2 | Tutor availability, work-log and excuse forms on real API routes

Branch: s1/m3/tutor-forms | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1 and initial API contracts

Goal: As a tutor, I want to record busy times, work logs and excuses through my own authenticated routes so that my records stay accurate.

Acceptance checks

- Tutor workflow endpoints derive ownership from the token and ignore browser-supplied tutorId values.
- Invalid or impossible time/hour values are rejected with clear field messages.
- Valid actions persist through the handwritten Express API and Prisma.

Member tasks

- [] Maintain availability/time-slot form and validation.
- [] Maintain work-log and excuse submission forms.
- [] Test valid, invalid, duplicate/state and cross-user protection cases.

Handoff: Member 5 receives the payload shapes; Member 6 receives required tables and constraints.

S1-M3-3 | Tutor UI contract, responsiveness and accessibility tests

Branch: s1/m3/tutor-ui-tests | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1, S1-M3-2 and S1-M4-2

Goal: As the team, we want regression tests around the tutor journey so that later changes do not break the working dashboard and forms.

Acceptance checks

- Every required tutor control remains keyboard reachable.
- Required actions are not clipped at the agreed narrow/mobile width.
- Contract or validation changes that break the current tutor journey fail tests.

Member tasks

- [] Keep dashboard/form component tests current.
- [] Cover labels, focus order and responsive layout.
- [] Run the shared test/coverage gate before handover.

Handoff: Give the Project Coordinator the test command/output and give Member 5 any contract failures as bugs.

Sprint 1 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M4-1 | Student overflow protected UI shell (Sprint 1 mock boundary)

Branch: s1/m4/student-overflow | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2

Goal: As a student, I want a clear protected overflow list so that I can understand unclaimed work while the real Student API is completed later.

Acceptance checks

- Only STUDENT users can open the page.
- Open, closed, empty and failed-load mock states are clear.
- The browser uses the agreed mock/API boundary and never talks directly to Neon.

Member tasks

- ☐ Maintain the Student overflow route/list/cards.
- ☐ Preserve open/closed/empty/error states and phone layout.
- ☐ Keep the mock contract stable until Sprint 2 connects the real backend endpoint.

Handoff: Give Member 5 and Member 6 the exact work-item fields needed by the UI.

S1-M4-2 | Shared role navigation and status components

Branch: s1/m4/shared-navigation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role map from S1-M1-2

Goal: As any Tutor Mx user, I want consistent role navigation and status messages so that the app feels like one product.

Acceptance checks

- Only navigation permitted for the signed-in role is shown.
- Loading, empty, error, success and confirmation states use consistent accessible components.
- Sign out is reachable from Organiser, Tutor and Student workspaces.

Member tasks

- ☐ Maintain shared navigation/status components.
- ☐ Keep visible focus and skip-to-content behavior.
- ☐ Test role visibility and responsive navigation.

Handoff: Members 2 and 3 consume these components; Member 1 supplies the authoritative role list.

S1-M4-3 | Student volunteer confirmation and shared UI tests (Sprint 1 mock boundary)

Branch: s1/m4/volunteer-confirmation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S1-M4-1 and S1-M4-2

Goal: As a student, I want a clear confirm/cancel volunteer interaction so that duplicate or accidental claims are prevented.

Acceptance checks

- Confirmation repeats the selected work item and Cancel sends no request.
- Pending state blocks duplicate submission and success/failure is explained.
- Sprint 1 behavior remains on the agreed mock boundary until Sprint 2 real endpoints are ready.

Member tasks

- ☐ Maintain confirm/cancel and pending states.
- ☐ Test duplicate, already-claimed and failure paths.
- ☐ Hand the real request/error expectations to Sprint 2 backend work.

Handoff: Member 5 receives expected duplicate/error responses; the Project Coordinator receives test evidence.

Sprint 1 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M5-1 | Separate handwritten Express backend and health endpoint

Branch: s1/m5/backend-foundation | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Chosen backend runtime and repository structure

Goal: As the team, we want a separately runnable handwritten API so that browser code never depends on direct database access.

Acceptance checks

- Backend starts independently and GET /health returns a documented safe response.
- Unhandled routes/errors use the shared safe error shape.
- Environment validation/logging never exposes secrets.

Member tasks

- ☐ Maintain the Express server, central error handling and health/readiness checks.
- ☐ Keep backend lint/test/build scripts working from the repository root.
- ☐ Cover health and error behavior with tests.

Handoff: Give Member 6 the environment and database-connection requirements; give all frontend Members the base URL.

S1-M5-2 | Auth middleware and core profile/course/tutor/tutor-workflow routes

Branch: s1/m5/core-api-contracts | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S1-M5-1, auth contract from Member 1 and schema from

Goal: As the application, we want protected handwritten endpoints so that all role data reaches React through one authorised API.

Acceptance checks

- Bearer verification and role/ownership middleware protect core routes.
- Course/tutor organiser routes and own-tutor dashboard/forms use Prisma on the backend only.
- Tutor workflow routes derive identity from the token, not a browser tutor ID.

Member tasks

- ☐ Maintain the current-profile, course, tutor and tutor-workflow routes.
- ☐ Keep success/error JSON and status codes consistent.
- ☐ Add success, validation, 401, 403, 404 and conflict tests when contracts change.

Handoff: Members 1-4 use the published route/payload examples; Member 6 aligns migrations and seed data.

S1-M5-3 | Three allocation-rule functions and boundary tests

Branch: s1/m5/allocation-rules | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Threshold decisions from client/coordinator and Member 6 data

Goal: As an organiser, I want server-side mark, clash and weekly-hour rules so that invalid allocations cannot be bypassed by the UI.

Acceptance checks

- Mark threshold and exact weekly limit follow the agreed inclusive boundaries.
- Touching time ranges are not a clash; true overlaps are.
- Missing/malformed data returns controlled unknown/failure rather than silent approval.

Member tasks

- ☐ Maintain pure rule functions and stable reason codes.
- ☐ Keep boundary tests for threshold, overlap and exact/exceeded hours.
- ☐ Ensure UI consumes server results instead of duplicating business logic.

Handoff: Member 2 maps reason codes to UI text; Member 6 provides candidate/availability/hour data.

Sprint 1 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M6-1 | Neon PostgreSQL schema, Prisma migrations and safe seed data

Branch: s | Reviewer: assigned reviewer | 4 points | Integrates to: Gitea main

Depends on: Field lists from Members 1-5

Goal: As the team, we want repeatable migrations and safe demo data so that every environment has the same data rules.

Acceptance checks

- Migrations create the required role/profile/course/mark/availability/hour/allocation structures from a clean database.
- Constraints reject invalid relationships and values.
- Seed data is invented/non-personal and no DATABASE_URL is committed.

Member tasks

- [] Maintain versioned Prisma migrations and seed data.
- [] Run schema/database verification after changes.
- [] Coordinate new fields with API and UI owners before migration changes land.

Handoff: Member 5 consumes the backend database connection; Members 2-4 receive stable demo identifiers and fields.

S1-M6-2 | Gitea integration checks, public deployment and shared Sprint 1 URL

Branch: s1/m6/ci-codecov | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Test scripts from Members 1-5 and repository admin access

Goal: As the team, we want one tested Gitea main plus a public demo so that everyone works from the same source and can demonstrate the integrated system.

Acceptance checks

- Gitea main is the official shared source; members work on branches and the Integration Lead integrates approved work.
- The public frontend loads at tutor-mx.pages.dev and calls the hosted Express API rather than Neon directly.
- Local tests remain required; pushed branches use the shared Gitea Actions runners, and queued jobs are allowed to auto-process.

Member tasks

- [] Maintain Gitea workflow files and honest CI/runner evidence.
- [] Sync approved Gitea main to the GitHub deployment mirror when releasing.
- [] Build the frontend and manually upload the fresh frontend/dist folder to Cloudflare Production when required.

Handoff: All Members use the same test script names; the Integration Lead uses these checks as the merge gate.

S1-M6-3 | South African public-holiday API spike and fallback contract

Branch: s1/m6/external-api-spike | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Client/coordinator chooses a relevant service and confirms

Goal: As Tutor Mx, we want a safe external public-holiday source to support scheduling without making the core system depend on that service.

Acceptance checks

- The backend-only spike returns a safe subset of South African holiday data.
- Timeout/unavailable/invalid responses return a controlled fallback.
- No user identity, token or database secret is sent to the external service.

Member tasks

- [] Maintain the Nager.Date holiday adapter/spike contract.
- [] Keep mocked success/timeout/invalid-response tests deterministic.
- [] Carry production route/display integration into later sprints rather than redoing the spike.

Handoff: Member 5 will integrate the adapter into the handwritten API; Member 4 will later display the safe response/fallback.

Sprint 1: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 1 SPECIFIC RELEASE CHECK

Verify new Student/Tutor/Organiser account creation -> Auth0 -> automatic onboarding -> correct role dashboard. Verify Organiser Tutor Management contains no Auth0-subject input.

Sprint 2: plan, order and handovers

26 August-15 September 2026 | Complete Basic journeys without rebuilding Sprint 1 foundations

DO NOT REPEAT EARLIER SPRINTS

Sprint 2 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Finish the remaining Basic end-to-end gaps: Student real API, allocation writes, approvals/statuses and hardening.
- Reuse the working automatic account linking and registered-tutor management from Sprint 1.
- Run representative-user and three-role regression before release.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 2 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M1-1 | Auth lifecycle hardening and email-verification recovery

Branch: s2/m1/account-lifecycle | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Sprint 1 auth foundation and deployed backend

Goal: As a Tutor Mx user, I want edge cases around verification, reset, sign-out and deletion handled reliably without repeating the working Sprint 1 onboarding flow.

Acceptance checks

- A linked existing account never returns to role choice after normal sign-in.
- An unverified email gets a clear retry/recovery path; add resend-verification support if the agreed Auth0 approach allows it.
- Reset/deletion/network failure cannot leave the app in a false signed-in or half-deleted state.

Member tasks

- [] Keep Sprint 1 self-service onboarding unchanged unless a defect is proven.
- [] Harden verification/retry, reset, deletion and post-action session cleanup.
- [] Add integration tests for failure/recovery paths.

Handoff: Members 2-4 receive a stable signed-in session/role API; Coordinator receives acceptance evidence.

S2-M1-2 | Role and ownership protection across the completed Basic API

Branch: s2/m1/role-protection | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S2-M1-1 and Member 5 Basic endpoints

Goal: As the school, we want every real Basic endpoint checked for role and ownership so that hidden navigation can never be bypassed.

Acceptance checks

- Organiser-only writes return 403 for Tutor/Student callers.
- Own-data routes do not reveal another Tutor/Student when identifiers are manipulated.
- Frontend route guards and backend authorisation remain consistent.

Member tasks

- [] Create an endpoint-to-role/ownership matrix for the now-real Basic routes.
- [] Add 401/403/cross-user tests for new Sprint 2 endpoints.
- [] Fix permission defects without moving security logic into React.

Handoff: Member 5 receives failing endpoint cases; Members 2-4 use the shared permission/error behavior.

S2-M1-3 | Cross-role security smoke suite and assigned fixes

Branch: s2/m1/security-smoke | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All Sprint 2 Basic journeys integrated on Gitea main

Goal: As the Integration Lead, we want a repeatable three-role smoke suite so that integration cannot break access or expose another role's data.

Acceptance checks

- Separate Student/Tutor/Organiser test accounts land only in their expected workspace.
- Critical endpoints show consistent 401/403 behavior for missing, expired and wrong-role tokens.
- Any severity-one/two security defect blocks release until fixed or formally recorded as a blocker.

Member tasks

- [] Create/run the cross-role smoke sequence against the shared environment.
- [] Open reproducible bugs for failures and fix Member 1-owned defects.
- [] Attach local test evidence plus the completed Gitea Actions run link/status; if queued, wait for the shared runner.

Handoff: Integration Lead runs this before the Integration Lead release decision; Coordinator links results into testing evidence.

Sprint 2 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M2-1 | Organiser course/tutor management hardening - registered tutors only

Branch: s2/m2/organiser-crud | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 organiser endpoints and Member 6 Basic schema

Goal: As an organiser, I want the working Sprint 1 management screens hardened around real registered Tutor accounts instead of rebuilding tutor creation.

Acceptance checks

- Tutor search/list comes from self-onboarded Tutor profiles and never asks for Auth0 subject/User ID.
- Edit/validation/refresh behavior stays correct after API failures.
- Course CRUD and tutor details/hour-limit/marks/busy-time edits persist and refresh cleanly.

Member tasks

- ☐ Do not rebuild Sprint 1 Tutor Management or add a manual tutor-create form.
- ☐ Fix validation, empty/error states, responsive layout and stale-data issues found in shared testing.
- ☐ Add integration tests around real course/tutor edit flows.

Handoff: Member 5 fixes contract mismatches; Member 6 validates data constraints.

S2-M2-2 | Live allocation create, edit and remove with server checks

Branch: s2/m2/allocation-board-basic | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S2-M2-1, Member 5 allocation endpoints and Member 6 queries

Goal: As an organiser, I want the Sprint 1 allocation placeholder replaced by real save/edit/remove behavior while keeping all rule checks visible.

Acceptance checks

- A valid candidate can be saved through a real allocation-write endpoint.
- Failed mark/clash/hour rules are rejected server-side and remain visible in the UI.
- Editing/removing an allocation updates hours and assignments without manual database edits.

Member tasks

- ☐ Connect real allocation create/edit/remove endpoints.
- ☐ Refresh candidate hours and assignment data after every successful change.
- ☐ Test each failed rule plus valid create/edit/remove.

Handoff: Member 5 enforces rules on server; Member 3 sees resulting tutor obligations.

S2-M2-3 | Organiser timesheet and overflow approvals with tests

Branch: s2/m2/organiser-approvals | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 3 timesheets, Member 4 volunteer requests and

Goal: As an organiser, I want to approve or return tutor timesheets and approve or reject student volunteer claims so that Basic workflows can complete end to end.

Acceptance checks

- Pending timesheets and claims show the information needed for a decision.
- Approve/reject/return actions persist a clear status and update the responsible user view.
- Duplicate or stale decisions are rejected safely.

Member tasks

- ☐ Build approval queues/views on the Organiser workspace.
- ☐ Connect to the Sprint 2 workflow endpoints and confirmation states.
- ☐ Test valid, rejected, stale and API-failure cases.

Handoff: Members 3 and 4 consume the resulting statuses; Coordinator receives demo-ready accounts/data.

Sprint 2 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M3-1 | Tutor dashboard hardening and live assignment refresh

Branch: s2/m3/tutor-dashboard-api | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 5 own-dashboard endpoint and Member 2 allocations

Goal: As a tutor, I want the already-live Sprint 1 dashboard to stay accurate when allocations change and when my weekly limit is unset or updated.

Acceptance checks

- The dashboard shows only the signed-in tutor and refreshes after allocation changes.
- No-work and no-hour-limit states remain usable.
- Failed refresh preserves the last safe state and offers retry.

Member tasks

- [] Do not replace the working own-tutor API with a new tutor-ID flow.
- [] Add refresh/status handling for allocation changes.
- [] Expand dashboard tests for changed assignments, unset limits and retry.

Handoff: Member 2 confirms allocation changes; Member 5 fixes data/permission contract issues.

S2-M3-2 | Complete timesheet submission workflow from existing work logs

Branch: s2/m3/timesheet-basic | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1 and Member 5 work-log/timesheet endpoints

Goal: As a tutor, I want my existing work logs to become a submit/review timesheet workflow so that organisers can approve actual worked hours.

Acceptance checks

- Draft work logs can be grouped/submitted into the agreed timesheet state.
- Invalid hours, duplicate submissions and wrong-session ownership are rejected.
- Tutor sees pending/approved/returned status after organiser action.

Member tasks

- [] Build submit/review-status UI on top of Sprint 1 work-log creation.
- [] Connect real timesheet submit/status endpoints.
- [] Test draft, submit, duplicate, returned and approved cases.

Handoff: Member 2 consumes submitted timesheets; Member 6 validates totals/status constraints.

S2-M3-3 | Excuse status workflow and Tutor Basic regression

Branch: s2/m3/excuse-regression | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1, S2-M3-2 and Member 5 excuse endpoint

Goal: As a tutor, I want the Sprint 1 excuse submission to show its later decision/status and remain reliable with the rest of the Basic journey.

Acceptance checks

- Submitted excuses show pending and later organiser outcome.
- Past/unowned sessions cannot be excused.
- Tutor regression covers dashboard, availability, work log, timesheet and excuse paths together.

Member tasks

- [] Add excuse status/history display rather than recreating submission.
- [] Connect organiser decision outcome once the endpoint exists.
- [] Run the full tutor Basic regression and fix reproducible defects.

Handoff: Coordinator receives a complete tutor demo script result; Member 2 sees excuse/availability impact.

Sprint 2 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M4-1 | Connect Student overflow to real open-work API

Branch: s2/m4/overflow-api | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 overflow endpoint and Member 6 schema

Goal: As a student, I want the Sprint 1 overflow shell to load real open work so that the mock boundary is removed without changing the established UI behavior.

Acceptance checks

- Only eligible/open work is returned for the signed-in Student.
- Closed/claimed work cannot be volunteered for.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Replace the Sprint 1 mock with the real student overflow endpoint.
- [] Keep the existing responsive list/cards and state components.
- [] Add integration tests for open, closed, empty and failure cases.

Handoff: Member 5 fixes response/permission issues; Member 6 supplies reproducible open/closed seed cases.

S2-M4-2 | Volunteer request and outcome statuses

Branch: s2/m4/volunteer-basic | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S2-M4-1 and Member 5 volunteer endpoint

Goal: As a student, I want my confirmed volunteer request stored and its outcome shown so that I know whether I received the work.

Acceptance checks

- Confirm creates one request and duplicate clicks/claims are controlled.
- Pending/approved/rejected/already-claimed statuses are visible.
- Another Student cannot view or mutate the request.

Member tasks

- [] Connect the Sprint 1 confirmation UI to the real volunteer endpoint.
- [] Show persisted request/outcome states.
- [] Test duplicate, race, rejected and permission cases.

Handoff: Member 2 consumes pending claims; Member 5/6 enforce uniqueness and atomic outcome.

S2-M4-3 | External-result UI, accessibility and Student Basic regression

Branch: s2/m4/shared-quality | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 6 external adapter and all Basic shared UI

Goal: As a user, I want the approved external holiday result/fallback and Student journey presented accessibly without weakening the working shared UI.

Acceptance checks

- External information is clearly supporting information and a safe fallback appears when unavailable.
- Student primary actions remain keyboard/mobile accessible.
- Student regression covers real overflow and volunteer flows.

Member tasks

- [] Render only the safe external API response/fallback from the backend.
- [] Run accessibility/responsive checks on Student and shared components.
- [] Fix Member 4-owned Basic regressions found in representative-user testing.

Handoff: Coordinator receives accessibility/test evidence; Member 6 receives any adapter failure cases.

Sprint 2 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M5-1 | Allocation-write and organiser-approval API completion

Branch: s2/m5/organiser-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic schema and Member 1 role matrix

Goal: As the application, we want real allocation mutations and organiser approval routes so Sprint 2 can complete workflows without redoing Sprint 1 course/tutor endpoints.

Acceptance checks

- Allocation create/edit/remove rechecks mark, clash and hours on the server inside safe database operations.
- Timesheet/claim organiser decisions enforce allowed state transitions.
- Legacy POST /api/tutors is not required by the normal self-service Tutor UI.

Member tasks

- [] Implement allocation mutation routes and transaction/service logic.
- [] Implement organiser timesheet/claim decision routes.
- [] Add validation, permission, conflict and state-transition tests.

Handoff: Member 2 connects the board; Member 6 tunes queries/constraints.

S2-M5-2 | Student overflow/volunteer and timesheet-status API

Branch: s2/m5/tutor-student-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic workflow schema and Member 1 ownership rules

Goal: As the frontend, we want the missing Student and workflow endpoints so the remaining Sprint 1 mocks/status gaps can become real.

Acceptance checks

- Student open-work and volunteer routes are role/ownership safe.
- Timesheet submit/status and excuse decision/status contracts are stable.
- Duplicate/race/state conflicts use safe 409-style responses.

Member tasks

- [] Implement Student overflow/volunteer endpoints.
- [] Complete timesheet/excuse status routes needed by Tutor/Organiser UI.
- [] Add success, validation, 401/403/404/409 tests.

Handoff: Members 3 and 4 connect their screens; Member 6 enforces database uniqueness/transitions.

S2-M5-3 | API contract, deployment and coverage hardening

Branch: s2/m5/api-quality | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S2-M5-1, S2-M5-2 and CI from Member 6

Goal: As the team, we want the expanded Sprint 2 API to keep the existing safe error contract and remain deployable/testable as one service.

Acceptance checks

- New routes use the established { error: { code, message } } shape and do not leak stack traces/secrets.
- Hosted health/smoke behavior stays stable after deployment.
- Coverage includes critical success, permission, validation and conflict branches.

Member tasks

- [] Standardise only new/changed routes; do not rewrite already-stable Sprint 1 contracts unnecessarily.
- [] Run backend build/smoke/coverage and fix real regressions.
- [] Provide deployment and API evidence to the Integration Lead.

Handoff: Integration Lead uses health/tests/Codecov as merge evidence; Coordinator uses route examples for API docs.

Sprint 2 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M6-1 | Sprint 2 migrations for workflow state and allocation writes

Branch: s2/m6/basic-workflow-schema | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Final Basic payloads from Members 2-5

Goal: As the team, we want only the new Sprint 2 data/state additions migrated so the working Sprint 1 schema is extended rather than recreated.

Acceptance checks

- New allocation mutation, timesheet/approval, excuse-status and volunteer-request data is represented with safe constraints.
- Migrations apply from the existing Sprint 1 schema and from a clean database.
- No manual production table edits are required.

Member tasks

- ☐ Add minimal versioned migrations for new Sprint 2 states/relations.
- ☐ Update seed data for approval and race/conflict cases.
- ☐ Run db verification and coordinate schema changes with API owners.

Handoff: Member 5 connects transactions/queries; Members 2-4 use stable demo data.

S2-M6-2 | Production public-holiday adapter with graceful fallback

Branch: s2/m6/external-api-adapter | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Approved Sprint 1 external service and Member 5 backend

Goal: As Tutor Mx, we want the Sprint 1 holiday spike exposed safely through the backend with caching/fallback so the UI can use it without repeating the research spike.

Acceptance checks

- Only the backend calls the external service and returns the approved safe fields.
- Timeout/unavailable/invalid-response uses the established fallback.
- The integration does not block core Tutor Mx workflows.

Member tasks

- ☐ Turn the validated Sprint 1 spike into a production backend adapter/route.
- ☐ Add sensible caching and deterministic mocked tests.
- ☐ Document the fallback and any service limits.

Handoff: Member 5 exposes the adapter through handwritten API; Member 4 renders success/fallback.

S2-M6-3 | Allocation data performance, deployment smoke and Codecov reliability

Branch: s2/m6/data-delivery-quality | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S2-M6-1, Member 5 API and Gitea Actions

Goal: As the Integration Lead, we want the completed Basic data flow and release checks reliable enough for repeated integration.

Acceptance checks

- Allocation candidates/writes and dashboard refresh use efficient queries with correct results.
- Clean setup/build/smoke works against the shared environment.
- Gitea Actions/coverage runs on the shared lecturer runners; queued jobs are expected and completed run evidence is retained.

Member tasks

- ☐ Profile/fix obvious allocation query issues and database constraints.
- ☐ Run clean migration/seed/check/smoke before release.
- ☐ Maintain Gitea CI/coverage config, verify the shared-runner result, and never invent green CI evidence.

Handoff: Integration Lead uses this as Sprint 2 merge gate; Coordinator receives run links and screenshots.

Sprint 2: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 2 SPECIFIC RELEASE CHECK

Verify real Student overflow/volunteer, allocation create/edit/remove and organiser approval/status journeys end to end. Confirm no Sprint 1 manual-ID flow was reintroduced.

Post-Sprint 2 production stabilisation

15-16 September 2026 | Live feedback fixes that become the Sprint 3 baseline

HISTORICAL RECORD RULE

Sprint 1 and Sprint 2 issue cards remain the record of what was planned and delivered in those sprints. The items below were discovered through integrated/live testing after the Sprint 2 review window and are recorded as the baseline Sprint 3 inherits.

Why this work was needed

- The merged product worked end to end, but some cross-layer behaviour was confusing or inconsistent when tested as a real Tutor or Organiser.
- The team chose narrow fix branches with regression tests rather than reopening completed Sprint 1/2 foundations.
- The main themes were ownership (who sets a Tutor schedule/capacity), visibility (calendar/workload presentation), candidate API correctness and post-save state consistency.

Chronological changes

Date	Change	Result / new baseline
15 Sep	Final Sprint 2 stability pass	A failing organiser integration was reverted rather than shipped; passing cache/timesheet corrections were retained. Course deletion, organiser layout and timetable/candidate behaviour were retested.
15 Sep	Tutor-owned work preferences	Tutor sets weekly work capacity and Busy/Class/Study/Other unavailable periods. Organiser views this read-only.
15 Sep	Weekly calendar + detail resilience	Tutor and Organiser share a weekly schedule calendar. Marks and schedule load independently. Optional legacy time-slot fields no longer hide valid marks. Confusing UTC-week wording was removed.
16 Sep	Candidate API contract repair	The query already returned clashDetails, but server mapping omitted it. The mapping now includes clashDetails so the frontend validator accepts valid candidate responses.
16 Sep	Post-save self-conflict repair	The saved allocation ID is passed as excludeAllocationId during refresh so a newly created/edited confirmed allocation does not clash with itself. Genuine conflicts still block.
16 Sep	Empty-state clarification	Only tutors passing mark + availability + capacity are selectable. If none pass, the board states "No tutors currently available"; failed tutors remain read-only with reasons.

Sprint 3 carry-forward

- Add previous/current/next-week navigation to workload cards so organisers can inspect future weekly capacity and confirmed allocations.
- Build timetable import and recurring exceptions into the existing Tutor-owned calendar instead of creating a second schedule model.
- Build ranking/bulk/override only on top of the corrected candidate contract and server revalidation.

Sprint 3: plan, order and handovers

16-29 September 2026 | *Intermediate features on the stabilised Basic release*

DO NOT REPEAT EARLIER SPRINTS

Sprint 3 starts from the latest approved Gitea main after the 15-16 September stabilisation. Keep Auth0 onboarding, Tutor-owned schedule/capacity, shared weekly calendar, separate mark/availability/capacity checks and the corrected candidate refresh contract.

Sprint 3 priorities

- Make weekly workload planning navigable across past/current/future weeks before adding higher-level ranking and reports.
- Add Intermediate ranking, bulk/override, timetable import, reports, notifications and search on the corrected Basic contract.
- Keep the sequential Member 1 → 6 handover method agreed on 11 September. Within each Member, Branch 2 comes from Branch 1 and Branch 3 comes from Branch 2; only cumulative Branch 3 merges to main.
- Treat representative-user feedback and production defects as first-class regression inputs.

Member	Sprint 3 ownership / emphasis
1	Auth/security, notifications/search, first-use guidance and shared quality
2	Organiser weekly workload navigation, ranked candidates, bulk/override, reports
3	Tutor timetable import, recurring exceptions, timesheet workflow and Tutor regression
4	Student overflow + shared responsive/accessibility consistency
5	Week-aware candidate/allocation APIs, Intermediate workflow/search/report/notification API
6	Week-aware data queries, ranking/reminder engine, migrations and delivery resilience

EVERY MEMBER

Before handover: run local acceptance checks, npm run check, coverage where required, push the branch to Gitea, wait for shared-runner Actions to complete, and retain the run/Codecov evidence.

Sprint 3 - Member 1

Authentication, role security, notifications/search and shared release checks

CARRY FORWARD

Do not recreate role/Profile onboarding. First-use guidance must describe missing Tutor configuration (capacity/schedule/assignments), not ask an established account to choose a role again.

S3-M1-1 | Notification inbox and due reminders

Branch: s3/m1/notifications **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: Affected users receive assignment/status changes and due reminders without leaking other users' records.

Acceptance checks

- Assignment, timesheet, claim and status changes produce one relevant notification for the responsible user.
- Deep links open only items the signed-in role may view.
- Repeated reminder processing controls duplicates.

Member tasks

- [] Build inbox/unread/read state and deep links.
- [] Use existing role/ownership contracts; no client-side security shortcuts.
- [] Add duplicate and cross-user notification tests.

Handoff: Members 2-4 reuse notification components; Members 5-6 receive permission/duplication failures.

S3-M1-2 | Global search and first-use Tutor guidance

Branch: s3/m1/search-onboarding **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: Users can find permitted records and Tutors understand incomplete setup without repeating authentication onboarding.

Acceptance checks

- Search returns only authorised result types.
- Tutor guidance references weekly capacity, schedule and assignments when missing.
- Dismiss/resume guidance does not change role/Profile.

Member tasks

- [] Build grouped search states.
- [] Link guidance to the existing Tutor work-preferences/schedule UI.
- [] Test wrong-role and missing-setup cases.

S3-M1-3 | Shared performance, accessibility and feedback fixes

Branch: s3/m1/shared-quality-fixes **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: High-priority shared issues found through production/representative-user testing are fixed and regression-tested.

Acceptance checks

- Linked feedback issue has a reproducible acceptance test.
- Critical keyboard/focus/label failures are closed.
- One slow shared path is measured and improved or documented.

Member tasks

- [] Implement assigned shared feedback fixes.
- [] Run focused accessibility checks.
- [] Keep release evidence tied to integrated main.

Sprint 3 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

NEW BASELINE

The Basic organiser UI already has eligible/not-eligible sections, specific clash details, shared weekly Tutor calendars and post-save self-conflict protection. Sprint 3 extends those behaviours; it does not replace them.

S3-M2-1 | Week-aware workload view, ranked candidates and shortage visibility

Branch: s3/m2/ranked-candidates **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: Organisers can inspect Tutor workload by week and then rank only candidates that already pass hard rules.

Acceptance checks

- Previous / This week / Next navigation updates every Tutor card for the selected week.
- Weekly capacity stays the Tutor-configured limit; confirmed allocations and worked hours are counted only in the selected week.
- Eligible Tutors are ranked using documented inputs; ineligible Tutors remain non-selectable with exact reasons.
- Shortage amount/location is visible when required work cannot be filled.

Member tasks

- [] Add week navigation and selected-week labels to Tutor Management workload cards.
- [] Connect ranked candidate results without moving authoritative calculations into React.
- [] Test Monday/Sunday week boundaries, future confirmed allocations and capacity-unset state.

Handoff: Member 6 owns week/ranking calculations; Member 5 owns response stability.

S3-M2-2 | Bulk fill and reasoned organiser override

Branch: s3/m2/bulk-override **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: Safe obvious assignments can be previewed/confirmed in bulk while exceptional overrides remain explicit and auditable.

Acceptance checks

- Preview has no side effects and only proposes currently eligible assignments.
- Commit returns clear per-item results and preserves hard constraints unless an explicitly allowed override is used.
- Override requires a non-empty reason and must never bypass role/ownership checks.

Member tasks

- [] Build preview/select/deselect/confirm UI.
- [] Reuse current mark/availability/capacity reason display.
- [] Handle partial result/failure without stale candidate state.

S3-M2-3 | Hours, budget and distribution reports

Branch: s3/m2/organiser-reports **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: Organisers can distinguish configured capacity, allocated hours and worked hours over a chosen reporting period.

Acceptance checks

- Weekly workload report uses the same authoritative backend calculations as Tutor cards.
- No/partial data states state limitations.
- Keyboard-only users can operate common board/report actions.

Member tasks

- [] Build summary/report cards with explicit units and period.
- [] Do not recalculate server totals differently in the UI.
- [] Add keyboard and responsive regression.

Sprint 3 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

TUTOR OWNERSHIP IS NOW THE BASELINE

The Tutor owns weekly work capacity and unavailable schedule entries. Organisers only inspect them. Timetable import must populate the same Tutor-owned schedule/calendar and keep existing Add/Edit/Delete behaviour.

S3-M3-1 | Timetable import, recurring exceptions and term calendar

Branch: s3/m3/timetable-import **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: Tutors can import or paste timetable data into the existing weekly calendar and add recurring exceptions accurately.

Acceptance checks

- Preview parsed Busy/Class/Study entries before confirmation.
- Invalid rows do not corrupt existing schedule.
- Weekly recurrence honours start/end dates; ONCE entries show only in the correct week.
- Local display time is consistent with saved data.

Member tasks

- ☐ Build import/paste preview and row errors.
- ☐ Map confirmed imports to the existing TimeSlot model/calendar.
- ☐ Add recurring-exception/term controls without giving Organiser mutation rights.

Handoff: Member 6 owns parser/term data; Member 2 sees resulting clash information.

S3-M3-2 | Timesheet corrections, disputes and payroll export

Branch: s3/m3/timesheet-workflow **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: Returned timesheets can be corrected/resubmitted with history, disputes are traceable and approved periods can be exported.

Acceptance checks

- Previous values and return reason remain visible.
- Dispute reason/status routes to organiser.
- Export excludes unapproved work and uses agreed fields/totals.

Member tasks

- ☐ Build correction/dispute views and timeline.
- ☐ Connect export action and safe download state.
- ☐ Keep worked-hours reporting aligned with selected-week views.

S3-M3-3 | Tutor first-use guidance, notifications and regression

Branch: s3/m3/tutor-intermediate-tests **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: The mature Tutor journey explains capacity/schedule setup and remains stable after Intermediate features.

Acceptance checks

- Guidance links to work preferences and schedule, not role selection.
- Notifications deep-link only to permitted Tutor records.
- Regression covers weekly calendar, import, work logs, timesheets and excuses.

Member tasks

- ☐ Integrate guidance and notification components.
- ☐ Run Tutor responsive/accessibility checks.
- ☐ Add regression for capacity-unset and future-week data.

Sprint 3 - Member 4

Student overflow plus shared responsive and accessible interface

SHARED UI RESPONSIBILITY

The new weekly calendar, metric cards, eligible/not-eligible candidate sections and “No tutors currently available” state are now shared visual patterns that must remain responsive, keyboard-usable and understandable without relying on colour alone.

S3-M4-1 | Overflow posting, claim and organiser-approval UI

Branch: s3/m4/overflow-workflow **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: Open overflow work can be posted, claimed and approved without duplicate/stale state.

Acceptance checks

- Open/closed/claimed states are explicit.
- One pending claim per valid request path.
- Approval/rejection updates the correct user and notification.

Member tasks

- ☐ Build organiser post/edit/close UI.
- ☐ Complete student claim history.
- ☐ Use shared status/confirmation components.

S3-M4-2 | Shared schedule/candidate/search/notification states

Branch: s3/m4/shared-intermediate-ui **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: New Intermediate interactions use consistent accessible patterns across roles.

Acceptance checks

- Weekly calendar remains readable on desktop and mobile; page-level horizontal overflow is avoided.
- Candidate pass/fail/empty states use text/icons as well as colour.
- Search/notification/bulk states restore focus predictably.

Member tasks

- ☐ Audit calendar/card layouts at common widths.
- ☐ Extend shared status/live-region patterns.
- ☐ Keep long names/descriptions wrapping cleanly.

S3-M4-3 | Representative-user fixes and app-wide accessibility regression

Branch: s3/m4/usability-regression **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: Critical usability findings are fixed and re-tested across Student, Tutor and Organiser journeys.

Acceptance checks

- Representative role scripts complete without coaching.
- Feedback items link issue → PR → retest.
- No critical keyboard/focus/label/status failure remains.

Member tasks

- ☐ Implement assigned UX/accessibility fixes.
- ☐ Run app-wide checklist and responsive checks.
- ☐ Preserve the improved organiser workload/candidate clarity.

Sprint 3 - Member 5

Handwritten Express API, validation, business rules, security and API tests

CONTRACTS TO PRESERVE

Candidate responses must include clashDetails; mark, timetable and capacity remain separate; allocation creation revalidates rules in the transaction; post-save/edit queries may exclude the allocation being mutated but may not hide genuine conflicts.

S3-M5-1 | Week-aware ranking, bulk-fill and override API

Branch: s3/m5/allocation-intermediate-api **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: The organiser UI receives protected week-aware candidate/ranking/bulk endpoints with stable rule details.

Acceptance checks

- Candidate request accepts a selected week/session and returns deterministic eligibility + reasons.
- Bulk preview is side-effect free; commit revalidates inside safe database operations.
- Allowed override stores a reason and never bypasses authentication/ownership.

Member tasks

- [] Extend candidate/ranking contract rather than replacing it.
- [] Carry clashDetails and capacity details through server mappers.
- [] Keep excludeAllocationId narrowly scoped to the current create/edit refresh path.

S3-M5-2 | Intermediate workflow, search, report and notification API

Branch: s3/m5/intermediate-api **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: Protected endpoints support timetable import, workflow history, search, reports and notifications.

Acceptance checks

- Timetable/availability endpoints remain Tutor-owned.
- Reports define allocated vs worked hours consistently for a requested period.
- Search is role-filtered and limited/paginated.

Member tasks

- [] Implement timetable/import and workflow endpoints.
- [] Add weekStart/weekEnd report parameters where required.
- [] Use safe error shape and preserve independent mark/schedule failure handling.

S3-M5-3 | API performance, security and Intermediate regression

Branch: s3/m5/api-intermediate-quality **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: The expanded API stays stable under realistic data and malformed/unauthorised requests.

Acceptance checks

- Candidate/week queries avoid obvious N+1/unbounded behaviour.
- Null/legacy recurrence fields remain safe.
- Full Basic + Intermediate API regression passes.

Member tasks

- [] Measure critical allocation/week/search/report endpoints.
- [] Add input-size and role/ownership tests.
- [] Keep production error bodies safe and structured.

Sprint 3 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

DATA MODEL CONTINUITY

Do not create a second availability/capacity model. Extend the existing Profile/TutorHourLimit/TimeSlot/Allocation/WorkLog structures only where new Intermediate state truly requires it.

S3-M6-1 | Intermediate schema, recurrence and report data

Branch: s3/m6/intermediate-schema **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Intermediate workflows and reporting have stable migrated data without breaking existing schedule rows.

Acceptance checks

- Existing WEEKLY/ONCE TimeSlot data remains readable.
- Recurring end boundaries and labels/categories support calendar/import use.
- Indexes support selected-week allocation/work-log queries.

Member tasks

- [] Add only forward migrations required by new state/history.
- [] Keep old valid rows compatible.
- [] Create fixtures spanning Monday/Sunday boundaries and future weeks.

S3-M6-2 | Week-aware capacity, ranking and reminder engine

Branch: s3/m6/intermediate-engine **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Backend helpers calculate selected-week capacity, deterministic ranking, shortages and reminders.

Acceptance checks

- Weekly capacity is the Tutor-configured limit; allocated/remaining/worked values are period-specific.
- Confirmed allocations are counted only in the week containing the session.
- Ranking consumes hard-rule eligibility rather than overriding it.

Member tasks

- [] Implement pure/testable week boundary helpers.
- [] Implement ranking/shortage factors with named inputs.
- [] Keep reminder processing idempotent.

S3-M6-3 | Delivery resilience, data performance and Codecov gate

Branch: s3/m6/release-quality **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Sprint 3 release checks prove the week-aware data path and integrated product are stable.

Acceptance checks

- Critical database queries meet recorded targets or limitations are documented.
- External-service failure does not block core workflows.
- Coverage/CI policy remains honest and reproducible.

Member tasks

- [] Measure/tune allocation/week queries.
- [] Run database/integration failure tests.
- [] Capture Gitea Actions/Codecov evidence for approved main.

Sprint 3: integration and review gate

Release checks for the sequential Member 1 → 6 workflow

- Each Member starts only after the previous Member's cumulative Branch 3 is integrated into Gitea main.
- The Integration Lead reviews/tests only the cumulative Branch 3 for the main merge, then reruns affected regression on main.
- Week navigation must show correct current/future capacity, confirmed allocations and worked hours without changing the Tutor's configured weekly limit.
- Weekly calendar, mark/schedule independent loading, clashDetails response mapping and post-save self-conflict protection must all remain green.
- Ranked/bulk features may select only Tutors passing all hard rules; "No tutors currently available" remains the correct empty state when none pass.
- Run three-role public smoke, API health/security/CORS checks and the relevant deployment mirror/Cloudflare release steps only from approved main.

SPRINT 3 SPECIFIC RELEASE CHECK

Verify selected-week workload navigation, ranking/bulk/import/report/notification/search flows, the Tutor-owned calendar and all post-Sprint 2 production regressions before release.

Sprint 4: plan, order and handovers

30 September-11 October 2026 | Advanced scope, hardening and release

DO NOT REPEAT EARLIER SPRINTS

Sprint 4 starts from the approved Sprint 3 main. Preserve week-aware workload calculations, Tutor-owned schedules, hard-rule candidate eligibility, shared calendar UI and the sequential workflow. Advanced features must reuse these contracts.

- Implement demonstrable Advanced scope: scenarios, collaboration/conflict handling, swaps, locks, audit/restore and release resilience.
- Scenario and swap engines must use the same hard mark/availability/capacity rules as live allocation.
- Final evidence must prove the deployed system corresponds to the approved Gitea main release.

Member	Sprint 4 ownership / emphasis
1	Production auth/session, notification/search release smoke, final security regression
2	Draft scenarios, concurrent organiser conflict UI, organiser final quality
3	Constraint-checked swaps, Tutor edge-case hardening, Tutor final regression
4	Overflow race handling, shared visual/accessibility polish, Student/shared final regression
5	Advanced scenario/conflict/swap/audit API, security/performance hardening
6	Explainable allocation engine, audit/restore data, final release gate/deployment resilience

Sprint 4 - Member 1

Authentication, role security, notifications/search and shared release checks

S4-M1-1 | Production account and session hardening

Branch: s4/m1/auth-hardening **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: Final release access remains secure and understandable under expired/revoked/stale-session cases.

Acceptance checks

- Protected work requires re-authentication after expired/revoked session.
- Reset/delete/sign-out boundaries do not leave stale protected access.
- 401/403 responses remain consistent.

Member tasks

- [] Audit production-like account/session flows.
- [] Fix redirect/session edge cases.
- [] Review browser storage and remove sensitive custom persistence.

S4-M1-2 | Notification/search final fixes and release smoke

Branch: s4/m1/shared-release-smoke **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: Public role entry, search and notifications work on the exact release candidate.

Acceptance checks

- Clean role accounts reach the correct workspace.
- Missing/deleted deep-link targets show safe state.
- Role-restricted search never leaks hidden records.

Member tasks

- [] Run clean-browser smoke.
- [] Fix assigned Sev 1/2 shared defects.
- [] Add regression for each fix.

S4-M1-3 | Final auth/security regression and failure rehearsal

Branch: s4/m1/security-regression **Reviewer:** Member 5 **Integrates to:** Gitea main

Goal: The release decision uses recorded auth/role/ownership evidence, not verbal confidence.

Acceptance checks

- Full account and role matrix passes.
- Cross-user requests return no private body.
- One controlled auth/network failure recovers safely.

Member tasks

- [] Run full matrix.
- [] Rehearse controlled failure.
- [] Open/fix/re-test Member 1 defects.

Sprint 4 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

SCENARIOS MUST USE THE SAME RULES

Draft and live allocations must share the corrected mark/availability/capacity contract, selected-week workload calculation and self-conflict exclusion semantics. A scenario may not appear valid by using different rule logic.

S4-M2-1 | Draft allocation scenarios, compare and publish

Branch: s4/m2/scenario-publish **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: Organisers compare draft allocations before publishing live work.

Acceptance checks

- Drafts do not change live allocations.
- Comparison shows assignment, shortage and week-aware workload differences.
- Locked assignments remain unchanged.

Member tasks

- ☐ Build draft list/create/rename/delete and comparison UI.
- ☐ Show selected week/period for workload summaries.
- ☐ Clearly label draft vs live.

S4-M2-2 | Concurrent organiser presence and conflict resolution

Branch: s4/m2/concurrent-board **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: Concurrent edits are detected rather than silently overwriting each other.

Acceptance checks

- Presence/last-update information visible.
- Second stale save returns conflict.
- Refresh/compare/retry produces explicit final state.

Member tasks

- ☐ Send expected version on mutations.
- ☐ Build conflict UI.
- ☐ Preserve candidate/calendar state after refresh.

S4-M2-3 | Organiser performance, accessibility and final regression

Branch: s4/m2/organiser-final-quality **Reviewer:** Member 4 **Integrates to:** Gitea main

Goal: Complete organiser flows are fast, keyboard-usable and regression-tested.

Acceptance checks

- Board, weekly workload, ranking, reports and scenarios meet recorded targets or limitations.
- Keyboard-only core actions work.
- Basic/Intermediate/Advanced organiser regression passes.

Member tasks

- ☐ Measure bottlenecks.
- ☐ Run focus/label/status checks.
- ☐ Run final organiser regression including no-candidate and post-save cases.

Sprint 4 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

S4-M3-1 | Constraint-checked Tutor session swaps

Branch: s4/m3/tutor-swaps **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: Tutor swaps are proposed/approved only when the replacement satisfies the same hard allocation constraints.

Acceptance checks

- Mark, schedule/confirmed-allocation clash and selected-week capacity are all checked.
- Failure reason is visible and no live swap occurs.
- Approved/rejected outcomes are visible to both Tutors.

Member tasks

- [] Build swap proposal/reason UI.
- [] Reuse rule-result presentation.
- [] Integrate organiser approval/status.

S4-M3-2 | Harden Tutor, calendar and timesheet edge cases

Branch: s4/m3/tutor-hardening **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: Boundary dates, recurrence, hours and workflow transitions remain trustworthy at release.

Acceptance checks

- Week boundaries and recurrenceUntil behave correctly.
- Duplicate/invalid imports do not corrupt existing schedule.
- Timesheet/excuse actions follow allowed transitions.

Member tasks

- [] Run boundary checklist across capacity/calendar/import/work logs/timesheets/excuses.
- [] Fix assigned Sev 1/2 defects.
- [] Preserve history.

S4-M3-3 | Complete Tutor regression and payroll/export verification

Branch: s4/m3/tutor-final-tests **Reviewer:** Member 6 **Integrates to:** Gitea main

Goal: The full Tutor journey and payroll output match deployed behaviour.

Acceptance checks

- Basic/Intermediate/Advanced Tutor paths pass.
- Export totals match fixtures and exclude unapproved work.
- Cross-user/wrong-role requests are denied.

Member tasks

- [] Run automated + clean-browser Tutor regression.
- [] Verify export schema/totals.
- [] Test valid/invalid swaps and calendar/capacity boundaries.

Sprint 4 - Member 4

Student overflow plus shared responsive and accessible interface

S4-M4-1 | Overflow race handling and notification polish

Branch: s4/m4/overflow-hardening **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Competing claims and stale pages resolve to one truthful outcome.

Acceptance checks

- Only permitted winner/pending outcome exists.
- Stale submit shows conflict and refreshes.
- Notifications reach only affected users.

Member tasks

- ☐ Run two-session race tests.
- ☐ Fix stale/conflict UI.
- ☐ Remove duplicate/irrelevant notifications.

S4-M4-2 | Final shared responsive and accessible visual system

Branch: s4/m4/shared-polish **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Tutor MX presents consistent role screens at phone/tablet/desktop sizes.

Acceptance checks

- Weekly calendars remain usable without whole-page horizontal scrolling.
- Metric cards/candidate results/forms/tables are not clipped.
- Focus, labels and status announcements are correct.

Member tasks

- ☐ Audit navigation, cards, calendars, tables and forms.
- ☐ Run contrast/focus/label checks.
- ☐ Capture common-width evidence.

S4-M4-3 | Student/shared UI final regression

Branch: s4/m4/student-final-tests **Reviewer:** Member 2 **Integrates to:** Gitea main

Goal: Student and shared visual/state components remain usable after final integration.

Acceptance checks

- Student list/claim/status/notification paths pass.
- Slow/error/fallback states are understandable.
- Keyboard/common-width regression passes.

Member tasks

- ☐ Run final automated and clean-browser shared regression.
- ☐ Fix assigned defects.
- ☐ Recheck organiser calendar/candidate shared components for regressions.

Sprint 4 - Member 5

Handwritten Express API, validation, business rules, security and API tests

S4-M5-1 | Advanced handwritten API workflows

Branch: s4/m5/advanced-api **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: Scenario, conflict, swap and audit endpoints reuse the same protected allocation contracts.

Acceptance checks

- Draft/publish is authorised and transactional.
- Stale version returns conflict without overwrite.
- Swap rechecks hard constraints before live change.

Member tasks

- ☐ Implement scenario/diff/publish.
- ☐ Implement version/conflict/presence support.
- ☐ Implement swap and audit/restore endpoints.

S4-M5-2 | API security, design and deployment hardening

Branch: s4/m5/api-hardening **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: Final API protects state, logs and deployment boundaries.

Acceptance checks

- Malformed/oversized/duplicate/unauthorised requests are controlled.
- Logs/errors expose no secrets/private payloads.
- Rollback/deployment path remains usable.

Member tasks

- ☐ Audit auth/ownership/validation.
- ☐ Add limits/idempotency where required.
- ☐ Remove dev bypasses/unsafe logs.

S4-M5-3 | API load, availability and final regression

Branch: s4/m5/api-final-tests **Reviewer:** Member 1 **Integrates to:** Gitea main

Goal: Performance and regression evidence covers Basic, Intermediate and Advanced routes.

Acceptance checks

- Allocation/week/search/report/scenario endpoints meet targets or limitations are explicit.
- Full success/boundary/permission/failure regression passes.
- Dependency/database slow failure is controlled.

Member tasks

- ☐ Run load/timing checks.
- ☐ Fix high-impact bottlenecks.
- ☐ Run final API/security/concurrency regression.

Sprint 4 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

S4-M6-1 | Explainable school-wide allocation engine with locks

Branch: s4/m6/advanced-allocation-engine **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: The proposal engine respects the same hard rules and organiser locks used by live allocation.

Acceptance checks

- Proposal never breaks mark, timetable or selected-week capacity constraints.
- Locks remain unchanged.
- Chosen/not-chosen explanations name factors without pretending the ranking is objective truth.

Member tasks

- [] Implement deterministic proposal engine.
- [] Preserve locks and report unmet work.
- [] Generate structured explanations.

S4-M6-2 | Audit history, restore and production database recovery

Branch: s4/m6/audit-restore **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: Important mutations are traceable and recoverable without deleting history.

Acceptance checks

- Actor/time/action/before-after reference captured.
- Restore creates a new auditable change.
- Backup/recovery rehearsal returns required data consistently.

Member tasks

- [] Complete audit/version schema/hooks.
- [] Implement safe restore operations.
- [] Add indexes/retention choices.

S4-M6-3 | Final Gitea release gate, deployment mirror and integration resilience

Branch: s4/m6/final-release-gate **Reviewer:** Member 3 **Integrates to:** Gitea main

Goal: The final release proves approved Gitea main, deployment mirror and public services correspond.

Acceptance checks

- Local check/coverage/database/smoke evidence captured.
- Migration/external/health failure blocks release or uses controlled fallback/rollback.
- GitHub mirror and Cloudflare production artifact update only from approved Gitea main.

Member tasks

- [] Run clean setup/full tests/coverage/database verification/public smoke.
- [] Sync approved main to GitHub mirror and let Render deploy backend.
- [] Build fresh frontend/dist and deploy Cloudflare Pages; record release SHA.

Sprint 4: integration and review gate

Final release criteria

- Follow the same sequential Member 1 → 6 integration method; only cumulative Branch 3 merges for each Member.
- Run final Basic + Intermediate + claimed Advanced regression after Member 6 integration.
- Verify weekly workload navigation, Tutor-owned calendar, candidate contract, no-candidate state and post-save consistency still pass.
- Verify scenario/swap engines use the same hard allocation rules and selected-week capacity semantics as live allocation.
- Verify audit/restore/recovery evidence and controlled failure rehearsals.
- Verify Gitea main SHA matches the GitHub deployment mirror; verify Render is Live for that release and the fresh Cloudflare frontend serves the same release.

SPRINT 4 SPECIFIC RELEASE CHECK

The deployed public system must match approved Gitea main, and every claimed Advanced feature must be demonstrable without regressing the corrected Basic allocation/calendar behaviour.

Requirement-to-sprint traceability and official review reminders

Updated to reflect the stabilised production baseline

Requirement	Where it lives now
Separate frontend/backend + handwritten API	S1-M5 foundation → S2 workflow endpoints → post-S2 candidate contract fixes → S3/4 Intermediate/Advanced APIs.
Auth0 + automatic role onboarding	S1-M1 foundation; later sprints harden/retest and never recreate role/Profile onboarding.
Neon + Prisma	S1-M6 foundation → S2 workflow migrations → existing TimeSlot/TutorHourLimit/Allocation/WorkLog extended only for new state.
Organiser Tutor Management	Registered Tutor flow → post-S2 read-only Tutor schedule/capacity calendar → S3 week navigation/ranking/reporting.
Tutor own-data security	Tutor owns weekly capacity and Busy/Class/Study schedule; API derives Tutor identity from token.
Allocation correctness	Mark + timetable + weekly capacity hard rules; clashDetails preserved; post-save excludeAllocationId prevents self-conflict; failed Tutors are non-selectable.
Student workflow	S1 mock shell → S2 real overflow/volunteer → S3/4 workflow/race/accessibility hardening.
CI / tests / Codecov	Local gate + shared Gitea runners + retained evidence; regression tests added for each production defect.
Public deployment	Approved Gitea main → GitHub mirror → Render backend + fresh Cloudflare frontend; release SHA and smoke checks retained.

Official review / rubric reminder

Review area	Evidence to show
Version control	Traceable fix branches/PRs/commits, main integration and Actions runs.
Documentation	This updated site/handbook matches the current stack and production behaviour.
Work tracker / methodology	Sequential Sprint 3/4 ownership and focused post-Sprint 2 bug records.
Implementation / quality	Working role journeys plus schedule/capacity/candidate regressions, security and accessibility.
Deployment	Gitea/GitHub SHA alignment, Render Live/health and Cloudflare fresh build.

Final beginner memory aid

What to remember from 16 September onward

- **Gitea main** = official team code. GitHub = deployment mirror only.
- **Sprint 1-2 history stays historical.** The 15-16 September fixes are the baseline for Sprint 3.
- **Tutor owns work preferences.** Tutor sets weekly capacity and Busy/Class/Study schedule; Organiser views it read-only.
- **Weekly calendar** is shared between Tutor and Organiser. Free space means available.
- **Allocation eligibility** = mark passes AND timetable passes AND weekly capacity passes. Failed Tutors are not selectable.
- **Candidate API** must carry clashDetails. Mark/schedule panels should fail independently where safe.
- **After save/edit**, exclude the saved allocation from its own clash refresh; genuine existing conflicts still block.
- **Week navigation** in Sprint 3 should show current/future allocated, remaining and worked hours for the selected week while keeping the Tutor's weekly limit unchanged.
- **Sprint 3-4 workflow:** Member 1 → 6 sequentially; within each Member Branch 1 → Branch 2 → Branch 3; only cumulative Branch 3 merges to main.
- **Release:** approved Gitea main → GitHub mirror → Render backend → fresh Cloudflare frontend → smoke test and record the release SHA.

SOURCE NOTE

This edition preserves the supplied handbook through Sprint 2 and revises the later roadmap using the team-recorded production fixes and deployment/test results through 16 September 2026. The team remains responsible for checking evidence against Gitea, the deployed system and the course brief before submission.